

Javascript

Contenidos

Metodos de **String**

Metodos de **Array**

Metodos de **Object**

Metodos avanzados

Inspirado en el bootcamp de
Manz.dev todos los créditos a el.

Metodos

- Si sabemos el tipo de dato, sabemos que métodos (funciones) tiene
- Busca el bloque String de la [CheatSheet de Javascript](#)

```
const text = "Alonso";    // Es un string

text.length              // 5
text.at(2)               // "o" → también text[2]
text.at(-1)              // ???
text.repeat(3);          // "AlonsoAlonsoAlonso"

String(Number(text)).repeat(4) + " " + text
```

Metodos

- Si sabemos el tipo de dato, sabemos que métodos (funciones) tiene
- Busca el bloque String de la [CheatSheet de Javascript](#)

```
const text = "Alonso";    // Es un string
```

```
typeof text;              // "string"
```

```
text.constructor.name     // "String"
```

```
text.length               // 
```

```
text.repeat(5)            // 
```

Metodos

- Si sabemos el tipo de dato, sabemos que métodos (funciones) tiene
- Busca el bloque String de la [CheatSheet de Javascript](#)

```
const number = 42;
```

```
typeof number;           // "number"  
number.constructor.name  // "Number"
```

```
number.length            // ✗  
number.repeat(5)         // ✗
```

```
number.toExponential()   // ✔ "4e+0"  
number.toPrecision(4)    // ✔ "4.000"
```

String templates

- Puedes usar ambas opciones, pero se prefiere ser consistente
- Strings template permite: Interpolación de variables y multilinea
- Extensión [ES6 String HTML](#)

```
// En general, las Strings en JS se escriben con doble comilla:  
const name = "Alonso";  
const text = "Hola";  
  
// También se pueden escribir con comilla simple:  
const text = 'Hola';  
  
// En general, se usan comillas dobles, pero hay un caso más interesante
```

String templates

- Puedes usar ambas opciones, pero se prefiere ser consistente
- Strings template permite: Interpolación de variables y multilinea
- Extensión [ES6 String HTML](#)

```
// Sin string templates:  
const name = "Alonso";  
const text = "Hola, soy " + name;    // "Hola, soy Alonso"  
  
// Con string templates:  
const text = `Hola, soy ${name}`;    // "Hola, soy Alonso"
```

String templates

- Puedes usar ambas opciones, pero se prefiere ser consistente
- Strings template permite: Interpolación de variables y multilinea
- Extensión [ES6 String HTML](#)

```
// Esto es especialmente interesante, porque nos permitirá crear cosas más complejas:  
const message = "¡Hola a todos!";  
const html = `  
// O incluso colocarlo más legible:  
const html = /* html */`<div class="container">  
  <p>${message}</p>  
</div>`;
```

Posición y fragmentos

- Los métodos `.search()`, `.indexOf()` e `.lastIndexOf()` buscan posiciones (index)
- Los métodos `.substring()` e `.slice()` buscan y devuelven fragmentos
- Los métodos `.replace()` y `.replaceAll()` buscan y reemplazan fragmentos

```
//           ↓0   ↓5           ↓15           ↓29
const text = "Estamos en el curso de Multimedios";

text.search("curso");      // 14
text.indexOf("o");         // 5
text.indexOf("u", 6);      // 15
text.lastIndexOf("e");     // 29
text.lastIndexOf("z", 28); // -1 (no se encuentra)
```


Posición y fragmentos

- Los métodos `.search()`, `.indexOf()` e `.lastIndexOf()` buscan posiciones (index)
- Los métodos `.substring()` e `.slice()` buscan y devuelven fragmentos
- Los métodos `.replace()` y `.replaceAll()` buscan y reemplazan fragmentos

```
//           ↓0    ↓5           ↓15           ↓29
const text = "Estamos en el curso de Multimedios";

text.substring(14, 22);    // "curso"
text.substring(14);        // "curso de Multimedios"
text.slice(14, 22);        // Igual a substring
text.slice(14, -2);        // ✅ "curso de Multimedi"

// Los métodos .match() también buscan (más avanzados)
```

Posición y fragmentos

- Los métodos `.search()`, `.indexOf()` e `.lastIndexOf()` buscan posiciones (index)
- Los métodos `.substring()` e `.slice()` buscan y devuelven fragmentos
- Los métodos `.replace()` y `.replaceAll()` buscan y reemplazan fragmentos

```
//           ↓0    ↓5           ↓15           ↓29
const text = "Estamos en el curso de Multimedios";

// Ojo, no reemplaza, devuelve una copia (esto es importante)
text.replace("curso", "clase"); // Reemplaza sólo el primero
text.replaceAll("a", "i");      // Reemplaza todas las "a"
text.replaceAll(/[aeiou]/, "i"); // Con expresiones regulares
```

Separar y unir

- Los métodos `.split()` / `.join()` separar o unen (String → Array, y viceversa)

```
const text = "Estamos en el curso de Multimedios";

const words = text.split(" ");           // (6) ['Estamos', 'en', 'el', 'curso', 'de', 'Multimedios']
const fragments = text.split("o");       // (4) ['Estam', 's en el b', '', 'tcamp de Multimedios']

words.push("todos", "juntos");           // (8) ['Estamos', 'en', ..., 'de', 'Multimedios', 'todos', 'juntos']
const newText = words.join(" ");         // 'Estamos en el bootcamp de Multimedios todos juntos'
```

Comprobaciones

- Comprueba si un texto empieza/acaba por algo `.startsWith()` / `.endsWith()`
- Comprueba si un texto incluye un fragmento `.includes()`

```
const firstMessage = "yes";
const secondMessage = "no";
const thirdMessage = "yes, of course";

const isPositive = firstMessage.startsWith("yes"); // true
const isPositive = secondMessage.startsWith("yes"); // false
const isPositive = thirdMessage.startsWith("yes"); // true

// Ojo: mayúsculas/minúsculas, espacios, variaciones, etc...
```

Comprobaciones

- Comprueba si un texto empieza/acaba por algo `.startsWith()` / `.endsWith()`
- Comprueba si un texto incluye un fragmento `.includes()`

```
const text = "Estamos en el curso de Multimedios";

text.endsWith("Multimedios");    // true

text.includes("Multimedios");    // true
text.includes("curso");          // true
text.includes("Alonso");         // false
```

Manipulación de textos

- Mayúsculas y minúsculas con `.toLowerCase()` / `.toUpperCase()`
- Suprimir («afeitar») espacios con `.trimStart()` / `.trimEnd()` / `.trim()`

```
const text = "¿Has practicado flex y grid?";  
const message = "  ¿Todavía nada?  ";  
  
text.toUpperCase();    // '¿HAZ PRACTICADO FLEX Y GRID?'  
message.trimStart();   // '¿Todavía nada? '  
message.trim();        // '¿Todavía nada?'
```

Métodos matemáticos

`Math.random),`
`Math.floor),`
`Math=4=(floor)`

`Math-round)`



Métodos matemáticos

- Math tiene muchos [métodos matemáticos](#) → [CheatSheet JS](#)
Por ejemplo, redondeos, o uno de los más interesantes: `Math.random()` → Propuesta futura
- Rellenar con (...) con métodos `.padStart()` / `.padEnd()`

```
const lowNumber = 42.3;  
const highNumber = 42.8;
```

```
Math.floor(lowNumber);    // 42  
Math.floor(highNumber);   // 42  
Math.ceil(lowNumber);     // 43  
Math.ceil(highNumber);    // 43  
Math.round(lowNumber);    // 42  
Math.round(highNumber);   // 43
```


Métodos matemáticos

- Math tiene muchos [métodos matemáticos](#) → [CheatSheet JS](#)
Por ejemplo, redondeos, o uno de los más interesantes: `Math.random()` → Propuesta futura
- Rellenar con (...) con métodos `.padStart()` / `.padEnd()`

```
Math.random();           // 0.15893673829326427    (Entre 0 y 1, con 16 decimales)
Math.random() * 6;       // 5.28581815715711717    (Entre 0 y 6, con 16 decimales)
Math.ceil(Math.random() * 6); // 6              (Entre 1 y 6, sin decimales)

const hours = 3;
const minutes = 8;
const time = `${hours.padStart(2, "0")}:${minutes.padStart(2, "0")}`; // 03:08
```

Métodos de Array

- Muchos métodos son similares a los que vimos en String
- Otros permiten hacer cosas más relacionadas con Array

```
const letters = ["A", "B", "C", "D", "E"];

letters.length           // 5
letters.at(0)            // "A"  (también letters[0])
letters.at(-1)           // "E"  (desde el final)
letters[-1]              // undefined
letters.with(0, "Z")     // ["Z", "B", "C", "D", "E"];
```

Mutación de variables originales

- Algunos métodos ⚠️ mutan el dato original, ¡identifícalos!
Por ejemplo: `.push()`, `.pop()`, `.unshift()` o `.shift()`

```
const letters = ["A", "B", "C", "D", "E"];

letters.push("F");           // Devuelve 6, ⚠️ letters muta: ["A", "B", "C", "D", "E", "F"]
letters.push("G");           // Devuelve 7, ⚠️ letters muta: ["A", "B", "C", "D", "E", "F", "G"]
letters.pop();               // Devuelve "G", ⚠️ letters muta: ["A", "B", "C", "D", "E", "F"]

letters.unshift("0");        // Devuelve 7, ⚠️ letters muta: ["0", "A", "B", "C", "D", "E", "F"]
letters.shift();             // Devuelve "0", ⚠️ letters muta: ["A", "B", "C", "D", "E", "F"]
```

Objetos



Métodos de Objetos

- Un objeto en Javascript es una forma de organizar información
- Propiedades y métodos

```
const user = {  
  name: "Alonso",  
  life: 99,  
  power: 10,  
  
  talk: function() {  
    console.log("Hello!!");  
  }  
}
```

```
user.name           // "Alonso"  
user["name"]        // "Alonso"  
  
const prop = "name";  
user[prop]           // "Alonso"  
  
user.role = "Teacher"; // Se pueden añadir  
user.talk();           // En consola: "Hello!!"
```

Formato JSON

- Pasar a texto los datos de un objeto Javascript (con algunas limitaciones)
- Pasar de objeto a JSON y viceversa: `JSON.stringify()` y `JSON.parse()`

```
const user = {  
  name: "Alonso",  
  life: 99,  
  power: 10,  
  
  talk: function() {  
    console.log("Hello!!");  
  }  
}
```

```
const json = JSON.stringify(user);  
// '{"name":"Alonso","life":99,"power":10}'  
  
const json = JSON.stringify(user, null, 4);  
// Te lo da «beautiful» a 4 espacios  
  
JSON.parse(json);  
// { name: "Alonso", life: 99, power: 10 }
```

Métodos de Array

- Desestructuración: spread y rest
- Algunos ejemplos

```
const firstLetters = ["A", "B", "C", "D", "E"];

// Los sacas del array, y reestructuras:
const letters = [...firstLetters, "F"];

// ['A', 'B', 'C', 'D', 'E', 'F']
```

Métodos de Array

- Desestructuración: spread y rest
- Algunos ejemplos

```
let first = 1;  
let last = 99;  
let aux;
```

```
// Opción 1: Clásico
```

```
aux = first;  
first = last;  
last = aux;
```

```
// Opción 2: Reestructurando
```

```
[first, last] = [last, first];
```


Métodos de Array

- Desestructuración: spread y rest
- Algunos ejemplos

```
const letters = ["A", "B", "C", "D", "E", "F"];

const halfSize = Math.floor(letters.length / 2); // 3
const firstPart = letters.slice(0, halfSize);    // ['A', 'B', 'C']
const lastPart = letters.slice(halfSize);        // ['D', 'E', 'F']

const result = [...firstPart, "medio", ...lastPart];
// ['A', 'B', 'C', 'medio', 'D', 'E', 'F']
```

Métodos de Array

- Métodos para comprobar: `.includes()`, `.indexOf()` o `.lastIndexOf()`
- Métodos para fragmentos: `.slice()`,  `.splice()` o `.toSpliced()`

```
//           ↓0  ↓1  ↓2  ↓3  ↓4  ↓5  ↓6  ← Slice
const letters = ["A", "B", "C", "D", "E", "F"];

letters.includes("A");    // true
letters.includes("Z");    // false
letters.indexOf("B");     // 1
letters.indexOf("H");     // -1
letters.slice(1, 3);      // Devuelve ["B", "C"]
```

Métodos de Array

- Métodos para comprobar: `.includes()`, `.indexOf()` o `.lastIndexOf()`
- Métodos para fragmentos: `.slice()`,  `.splice()` o `.toSpliced()`

```
//           ↓0   ↓1   ↓2   ↓3   ↓4   ↓5   ← Splice
const letters = ["A", "B", "C", "D", "E", "F"];

letters.toSpliced(1, 3);
// Selecciona ["B", "C", "D"] y devuelve lo que queda: ['A', 'E', 'F']
// letters no muta, queda igual

letters.splice(1, 3);
// Selecciona ['B', 'C', 'D'], lo extrae y lo devuelve
//  letters muta a lo que queda: ['A', 'E', 'F']
```

Ordenar y clonar



Ordenar Array

- Métodos para invertir: ⚠️ `.reverse()` y `.toReversed()`
- Métodos para ordenar: ⚠️ `.sort()` y `.toLocaleSorted()`

```
const letters = ["A", "B", "C", "D", "E", "F"];

const inverseLetters = letters.toReversed();
// ['F', 'E', 'D', 'C', 'B', 'A']
// letters no muta

const inverseLetters = letters.reverse();
// ['F', 'E', 'D', 'C', 'B', 'A']
// ⚠️ letters muta: ["A", "B", "C", "D", "E", "F"]
```

Ordenar Array

- Métodos para invertir: ⚠️ `.reverse()` y `toReversed()`
- Métodos para ordenar: ⚠️ `.sort()` y `.toSorted()`

```
const letters = ["C", "Z", "A"];

letters.toSorted(); // ['A', 'C', 'Z']
letters.sort();     // ['A', 'C', 'Z'] ⚠️ letters muta: ['A', 'C', 'Z']

const numbers = [1, 5, 8, 10, 4, 25, 2];

numbers.toSorted(); // [1, 10, 2, 25, 4, 5, 8]    ???
```

Ordenar Array

- Métodos para invertir: ⚠️ `.reverse()` y `toReversed()`
- Métodos para ordenar: ⚠️ `.sort()` y `.toSorted()`

```
const numbers = [1, 5, 8, 10, 4, 25, 2];

numbers.toSorted();           // ordenación alfabética
// [1, 10, 2, 25, 4, 5, 8]

numbers.toSorted((a, b) => a - b); // ordenación numérica descendente
numbers.toSorted((a, b) => b - a); // ordenación numérica ascendente
letters.toSorted((a, b) => a.localeCompare(b)); // descendente ['A', 'C', 'Z']
letters.toSorted((a, b) => b.localeCompare(a)); // ascendente ['Z', 'C', 'A']
```

Sobre las copias de estructuras

- Aprende a diferenciar cuando es un valor y cuando una referencia
- Usa `...` para copias superficiales y `structuredClone()` para copias profundas

```
const user = {  
  name: "Alonso",  
  role: "teacher"  
}  
  
const copy = user;           // Parece que estamos haciendo una copia del objeto, ✗ pero no  
user.name === copy.name     // Son iguales, ambos "Alonso"  
  
copy.name = "Ivan";          // Cambiamos el nombre de la copia  
user.name === copy.name     // Son iguales, es decir, ambos "Ivan" ✗
```


Sobre las copias de estructuras

- Aprende a diferenciar cuando es un valor y cuando una referencia
- Usa `...` para copias superficiales y `structuredClone()` para copias profundas

```
const user = {  
  name: "Alonso",  
  techs: ["css", "js"]  
}  
  
const copy = { ...user }           // Reestructuramos los elementos en un nuevo objeto  
copy.name = "Paco Hdez";           // Cambiamos el nombre de la copia  
user.name === copy.name            // No son iguales, ✅ parece que la copia es correcta  
  
copy.techs[0] = "tailwind";  
user.techs[0] === copy.techs[0]    // Son iguales, ❌ volvemos a tener el mismo problema
```

Sobre las copias de estructuras

- Aprende a diferenciar cuando es un valor y cuando una referencia
- Usa `...` para copias superficiales y `structuredClone()` para copias profundas

```
const user = {  
  name: "Alonso",  
  techs: ["css", "js"]  
}  
  
const copy = structuredClone(user); // Copia profunda  
  
copy.techs[0] = "tailwind";  
user.techs[0] === copy.techs[0] // No son iguales, ✅ ahora si hizo copias a cualquier nivel
```

Iteradores de objetos

- Métodos para iterar propiedades y sus valores `.keys()` o `.values()`
- Métodos para convertir de objeto a entradas y viceversa `.entries()` o `.fromEntries()`

```
const user = {  
  name: "Alonso",  
  role: "teacher",  
  life: 99  
}
```

```
Object.keys(user)    // ['name', 'role', 'life']  
Object.values(user)  // ['Alonso', 'teacher', 99]
```

Iteradores de objetos

- Métodos para iterar propiedades y sus valores `.keys()` o `.values()`
- Métodos para convertir de objeto a entradas y viceversa `.entries()` o `.fromEntries()`

```
const user = {                // entries = [  
  name: "Alonso",            //   ['name', 'Alonso'],  
  role: "teacher",           //   ['role', 'teacher'],  
  life: 99                   //   ['life', 99]  
}                             // ]
```

```
const entries = Object.entries(user);  
const original = Object.fromEntries(entries);
```

Introducción a Array functions

- El método de array `.forEach()` ejecuta una función por cada elemento
- Volveremos a ello más adelante.

```
const letters = ["A", "L", "O", "N"];

function show(element) {
  console.log("Elemento: " + element);
}

letters.forEach(show);
```

```
const letters = ["A", "L", "O", "N"];

// Abreviado con arrow function
letters.forEach((el) => console.log(el));

// Elemento, posición, array
letters.forEach((el, index, array) => {
  console.log(`Elemento #${index}: ${el}`);
});
```

Array functions: `.forEach()` vs `.map()`

```
// No devuelve nada nunca (undefined)
animals.forEach((el, index) => {
  console.log(`Elemento #${index}: ${el}`);
});
```

```
// Transforma el array original en uno derivado
const sizes = animals.map((el, index, array) => {
  return el.length;
});
```

```
// Versión abreviada
const sizes = animals.map((el) => el.length);
```

Referencias

- [CheatSheet Javascript](#)
- [bootcamp.manz.dev](#)